

# Congress Trading Signal

## La couche données

Extraction, identification, enrichissement et validation honnête des déclarations  
boursières du Congrès américain — Chambre et Sénat, 2020–2026

Alice Lemaire · 27 juin 2026

---

## Résumé

Les membres du Congrès américain déclarent leurs transactions boursières sous le *STOCK Act* ; ces déclarations, exploitées par une stratégie de *copy-trading*, exigent au préalable une couche données propre, traçable et honnêtement validée. Ce rapport raconte la construction de cette couche, étape par étape — de la résolution d'identité jusqu'à la table finale enrichie. Le pipeline reconstruit **90 487 transactions** sur deux chambres et sept ans (2020–2026) : **81 646** pour la Chambre (32 676 électronique + 48 970 OCR) et **8 841** pour le Sénat (7 161 électronique + 1 680 papier OCR). Chaque transaction est rattachée à son auteur (**99,99 %** des lignes Chambre, **100 %** au Sénat), enrichie (ticker, secteur **GICS**→**ETF SPDR**, type d'actif, montant *midpoint*, ancienneté), aboutissant à la **table 12/12 champs**. La validation externe contre Quiver — **jamais réinjectée** — donne une concordance transaction-niveau de **85,9 %/an** (Chambre, un plancher) et **98–100 %/an** (Sénat), avec **9** et **0** vrais-absents après explication ; le papier scanné, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne. La correction est figée par un **golden octet-à-octet** (108 fichiers Chambre, 69 Sénat) et des reproductions fonction-par-fonction. Chaque étape est exposée selon le même fil : le **problème** concret, la **façon dont il est résolu** (avec la fonction réelle), et le **résultat chiffré**.

## 1 Introduction & contexte

Depuis le **STOCK Act** (2012), les membres du Congrès américain doivent divulguer publiquement leurs transactions sur titres dans un *Periodic Transaction Report* (PTR), en principe sous **45 jours**. L'intuition de recherche — documentée académiquement (Ziobrowski *et al.* 2004/2011, Karadas 2019) — est que ces personnes, par leur appartenance à des commissions clés (Finance, *Armed Services*, Intelligence) ou leur exposition à l'information, produisent par leurs déclarations un **signal potentiellement exploitable**. Une stratégie de *copy-trading* consiste à répliquer ces transactions *après* leur publication (anti-*look-ahead* : on date chaque transaction par sa date de divulgation, jamais par une date postérieure connue).

Avant toute stratégie, il faut une **couche données** irréprochable : extraire fidèlement les déclarations des deux chambres, les rattacher à la bonne personne, les enrichir (ticker, secteur, montant) et les valider honnêtement. C'est l'objet de ce rapport. La stratégie et le backtest associés constituent un travail de recherche séparé, hors du périmètre traité ici.

## 2 Sources & acquisition

**Principe anti-look-ahead.** Chaque transaction est datée par sa **date de divulgation** (*disclosure\_date*, la date de dépôt du PTR), jamais par une date postérieure connue : une stratégie ne peut copier qu'*après* publication.

**Chambre — House Clerk.** Un index XML par année (*{an}FD.xml*) liste les dépôts ; on filtre *FilingType=P* (les PTR) et on télécharge le PDF de chaque dépôt. Source publique, sans barrière. Les PDF se répartissent en **lisibles** (couche texte → parsing déterministe) et **scannés** (image → OCR).

**Sénat — eFD (*Electronic Financial Disclosure*).** Plus fermé : un **garde-fou CSRF**. Il faut

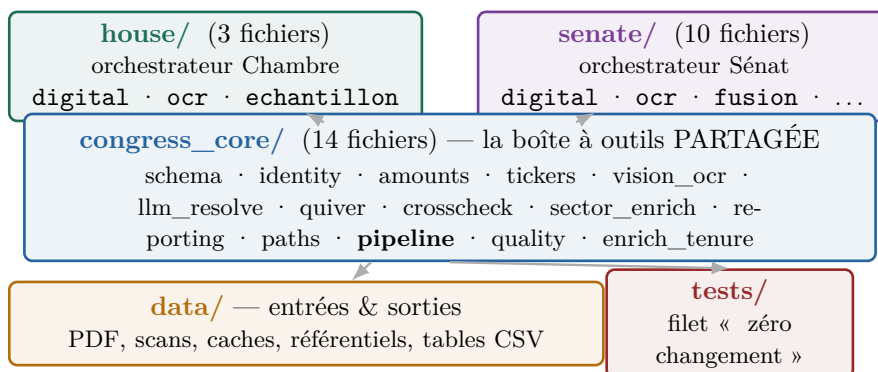
d'abord accepter un formulaire d'accord (`accept_agreement` : GET pour un `csrftoken`, puis POST `prohibition_agreement` avec `csrfmiddlewaretoken` et l'en-tête `X-CSRFToken`). On interroge ensuite l'API de recherche (`report_types=[11]` = PTR). Les déclarations électroniques sont en **HTML** ; les déclarations papier renvoient des **images .gif** servies par `efd-media-public.senate.gov`. On reste « poli » (pause entre requêtes, aucun contournement) et on **cache tout sur disque** — un re-run ne re-télécharge rien.

**Quiver = vérification externe *uniquement*.** Quiver Quantitative (agrégateur commercial) sert de **vérité-terrain indépendante**. Il n'est *jamaïs* réinjecté dans nos tables : on mesure si notre extraction concorde, on ne la complète pas. C'est une règle structurante du projet.

### 3 Architecture & méthodologie

#### 3.1 Trois couches de code + une couche de données

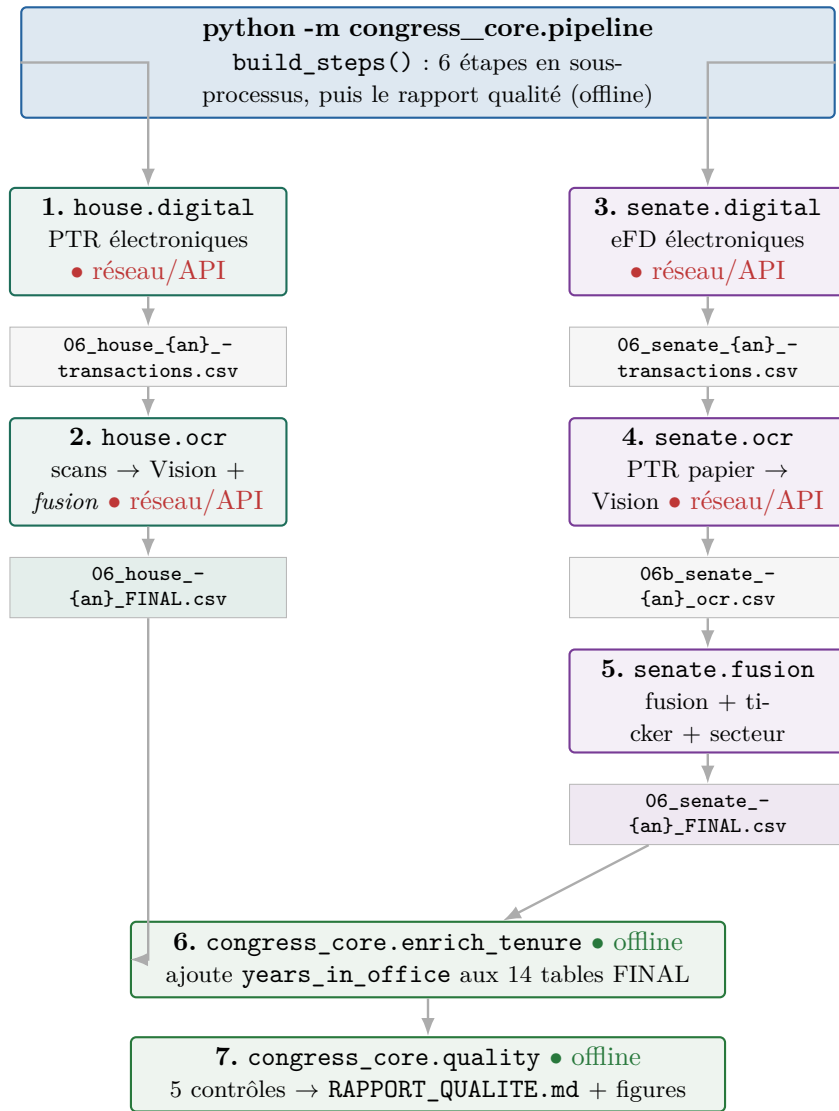
La structure est une **boîte à outils partagée** (`congress_core`, 14 modules) que **deux orchestrateurs** (un par chambre) utilisent, plus la donnée et les tests.



**L'idée directrice.** `congress_core` fournit les **ingrédients et ustensiles** (couper un nom en bioguide, lire un scan, calculer un secteur...), et `house/` comme `senate/` sont **deux recettes** qui s'en servent dans leur ordre. Elles ne diffèrent que parce que les **sources** diffèrent : la Chambre expose un index XML + des PDF du *Clerk* ; le Sénat un site eFD (HTML) + des images `.gif`. C'est cette asymétrie de sources qui explique que `senate/` compte plus de fichiers (scraping et OCR papier plus complexes).

#### 3.2 Le pipeline : un point d'entrée unique

`congress_core/pipeline.py` (fonction `build_steps`) enchaîne les modules en sous-processus. La commande `python -m congress_core.pipeline -years 2020-2026` assemble **6 étapes** d'orchestration ; un **7<sup>e</sup>** traitement — le rapport qualité (`congress_core/quality.py`) — se lance séparément, hors-ligne et en lecture seule.



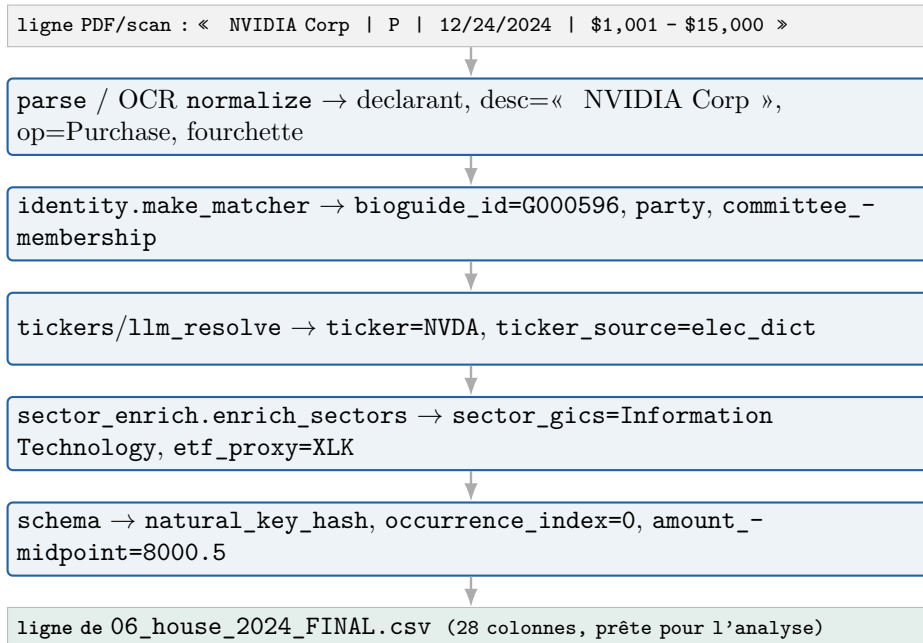
### 3.3 La convention de numérotation *est* l'enchaînement

Sur le disque, le préfixe d'un fichier = son étape dans le pipeline. C'est ce qui rend la couche données lisible sans documentation externe.

| Préfixe      | Étape                                                   | Écrit par                    |
|--------------|---------------------------------------------------------|------------------------------|
| 03_          | index des PTR (FilingType=P) dans la fenêtre            | house.digital                |
| 04_          | manifeste : lisible / non_lisible / absent              | house.digital                |
| 05_          | échecs de parsing                                       | *.digital                    |
| 06_          | table <b>digitale</b> (PTR électroniques)               | *.digital                    |
| 06b_         | table <b>OCR</b> (scans / papier)                       | *.ocr                        |
| 06c_         | échecs OCR                                              | *.ocr                        |
| 06_..._FINAL | <b>digital + OCR fusionnés</b> , 28 colonnes            | house.ocr /<br>senate.fusion |
| 07_...07f_   | comparaison / réconciliation <b>Quiver</b> (validation) | quiver / quiver_<br>audit    |
| 08_          | cross-check « semaine 1 » (Chambre)                     | house.digital                |
| 00_          | tableaux de bord (statut, concordance)                  | *.digital / fusion           |
| _préfixe     | caches & brouillons (_quiver_*_cache, _scan_census...)  | divers                       |

### 3.4 Le voyage d'une transaction (vue compacte)

Avant le récit détaillé (§4), voici la trajectoire d'une ligne, de la source à la table finale — en nommant la fonction à chaque étape. C'est elle qui servira de **fil rouge**.



## 4 Construction de la donnée, étape par étape

Cette section suit une transaction *dans l'ordre où le pipeline la construit* — du nom griffonné sur un formulaire jusqu'à la ligne enrichie de la table finale. Pour chaque étape : le **problème** concret, **comment on le résout** (avec la fonction réelle), et le **résultat** chiffré. Pour rendre le tout tangible, on suit un **fil rouge** : un achat NVIDIA déclaré par la représentante M. T. Greene (bioguide\_id=G000596), repris à chaque étape.

### 4.1 D'abord, savoir qui a déclaré : la résolution d'identité

**Le problème.** Une déclaration ne donne qu'un *nom libre* — « Hon. Earl L. Carter » ici, « Buddy Carter » là, « N. Pelosi » ailleurs. Le même élu apparaît sous plusieurs graphies au fil des années ; sans clé stable, on le compterait deux fois et l'on ne pourrait rattacher ni son parti ni ses commissions.

**Comment on le gère.** Le module `congress_core/identity.py` charge le référentiel public *congress-legislators* (load\_reference, en ligne avec repli sur des YAML embarqués), et en tire un index de noms volontairement *tolérant* : pour chaque élu, on indexe non seulement (nom, prénom) mais aussi le **surnom** (table de 31 équivalences : richard→rick, william→bill...), le second prénom, le premier mot du nom officiel et le dernier mot d'un nom composé. Le cœur est la cascade `make_matcher`, qui résout (nom, prénom) dans un ordre de priorité décroissante : (1) un **override manuel** pour les cas irréductibles (un seul est figé : « Nicholas V. Taylor » = Van Taylor) ; (2) la **correspondance exacte** sur l'index ci-dessus (surnoms et seconds prénoms compris) ; (3) à défaut, le **repli par nom de famille unique** (si un seul élu porte ce nom). `enrich_identity` applique le matcher puis ajoute party, committee\_membership et le drapeau de commission clé ; `add_years_in_office` calculera plus tard l'ancienneté. Côté Sénat, `senate/identity.py` ajoute une désambiguïsation par chambre/titulaire et **exclut les délégués** non-votants (sans quoi ils apparaîtraient « chez Quiver mais pas chez nous »).

**Le résultat.** 99,99 % des lignes Chambre (81 642 / 81 646) et 100 % du Sénat reçoivent un bioguide\_id. Les 4 lignes sans identifiant sont *une* déclaration manuscrite de 2023 (« Ada Norah Henriquez », doc 8219483). On compte **256 bioguides pour 275 noms** (Chambre) : l'écart, ce sont précisément les variantes orthographiques d'un même élu, ramenées au même identifiant — exactement ce que la résolution corrige.

*Fil rouge.* Sur notre ligne, le nom déposé « Marjorie Taylor Greene » est résolu par `make_matcher` en `bioguide_id=G000596`, ce qui attache aussitôt son parti et ses commissions.

## 4.2 Ensuite, trier les formats : électronique ou scanné ?

**Le problème.** Une déclaration arrive sous deux formes très différentes, qui n'appellent pas le même traitement : un **texte** exploitable directement (on peut le *parser*), ou une **image** qu'il faut « lire » par OCR. Se tromper de piste, c'est soit perdre des données, soit gaspiller des appels Vision sur un document déjà lisible.

**Comment on le gère.** Côté Chambre, `house/digital.py:build_manifest` ouvre chaque PDF avec `pdfplumber` et applique un test binaire *simple et déterministe* — `bool(text.strip())` : une couche de texte non vide  $\Rightarrow$  **lisible** (piste électronique) ; vide  $\Rightarrow$  **non\_lisible** (piste OCR) ; fichier absent  $\Rightarrow$  **absent**. Le verdict est écrit dans `04_download_manifest.csv` et **pilote tout le reste** ; sa simplicité même fait sa fiabilité (pas d'heuristique floue). Côté Sénat, la distinction vient de la liste eFD elle-même (`fetch_ptr_list`) : chaque dépôt porte un `kind` — `ptr` (déclaration HTML électronique) ou `paper` (image .gif scannée).

**Le résultat.** Ce tri d'apparence anodine est la fourche du pipeline : il explique la composition finale (Chambre 32 676 électronique + 48 970 OCR ; Sénat 7 161 électronique + 1 680 papier) et oriente chaque document vers la bonne des deux sous-sections suivantes.

*Fil rouge.* Le PDF de notre achat NVDA possède une couche de texte : il est classé **lisible** et part en piste électronique (§4.3).

## 4.3 La piste digitale de la Chambre (PDF lisibles)

**Le problème.** Sur un PDF Clerk lisible, les transactions ne forment pas un tableau propre : une *même* transaction peut s'étaler sur plusieurs lignes, le montant déborder sur la ligne suivante, le ticker se cacher entre parenthèses et le libellé de l'actif courir sur des lignes de continuation. Un simple « découpage par ligne » raterait la moitié des transactions.

**Comment on le gère.** `house/digital.py:run_year` orchestre la séquence : `load_ptr_index` lit l'index XML (`FilingType=P`, fenêtre de divulgation) et produit `03_ptr_index.csv` ; `parse_ptr` fait le gros du travail — il **reconstitue** d'abord les lignes coupées (`_join_split_lines`, qui recolle un montant scindé en deux), détecte chaque transaction par l'expression régulière `TXN_RE` (un code `P|S|E`, deux dates, une fourchette \$), puis remonte de quelques lignes pour rattacher la description d'actif et le ticker (entre parenthèses ou préfixe d'échange). `join_identity` rattache ensuite le bioguide et les métadonnées, et `finalize` calcule la clé naturelle, l'`occurrence_index` et déduplique. La sortie est `06_house_{an}_transactions.csv` ; les PDF lisibles ne donnant *aucune* ligne sont tracés dans `05_parse_failures.csv` — un échec est toujours visible, jamais silencieux.

**Le résultat.** **32 676** transactions électroniques, avec un taux de parsing très élevé (99–100 %).

*Fil rouge.* `parse_ptr` extrait de notre PDF : opération P (Purchase), `transaction_date` 24/12/2024, libellé « NVIDIA Corp », ticker NVDA capté entre parenthèses, fourchette \$1 001–\$15 000.

## 4.4 La piste digitale du Sénat (HTML eFD)

**Le problème.** Le Sénat n'expose pas de PDF : il faut d'abord franchir un **garde-fou CSRF** (un jeton qu'on n'obtient qu'en acceptant un formulaire d'accord), puis parser des **tableaux HTML** dont l'intitulé et l'ordre des colonnes *varient* d'une déclaration à l'autre.

**Comment on le gère.** `senate/digital.py:run_year` commence par `accept_agreement` (GET du `csrftoken`, POST `prohibition_agreement` avec l'en-tête `X-CSRFToken`), puis `fetch_ptr_list` interroge l'API de recherche (`report_types=[11]`) et `fetch_report` récupère chaque déclaration (avec cache disque). `parse_electronic` lit alors les tableaux via `pd.read_html` — qui renvoie une liste de *DataFrames* bruts — et c'est `_find_col` qui **retrouve les bonnes colonnes par appariement flou** (chercher une colonne dont le nom contient « ticker », ou « asset » et « name »), seule façon robuste face à des en-têtes irréguliers. L'opération est normalisée (`norm_op`) et le montant ramené à son milieu (`amount_midpoint`) ; l'identité, le ticker et la déduplication sont délégués à `senate/identity.py:enrich`.

Sortie : 06\_senate\_{an}\_transactions.csv.

**Le résultat.** 7 161 transactions électroniques pour le Sénat.

## 4.5 La piste scannée de la Chambre (OCR Claude Vision)

**Le problème.** Reste l'autre moitié : des **images**. À quoi ressemblent-elles ? Le census des 547 PDF scannés en distingue trois familles : **A — tapé droit** (74 docs), **B — tapé mais l'image est couchée** (322 docs, le gros du volume), **C — manuscrit** (151 docs). Or un modèle de vision lit mal une page tournée, et plus mal encore une date manuscrite.

**Comment on le gère.** Le moteur `congress_core/vision_ocr.py` **redresse d'abord**. L'idée naïve — demander au modèle « de combien faut-il tourner ? » — échoue (il répond presque toujours « 90 ») ; à la place, `detect_rotation` lui montre la page aux **4 orientations** (`ROT_CANDIDATES=[0,90,180,270]`) et lui fait *reconnaître* celle qui est droite — une tâche de reconnaissance, bien plus fiable qu'une estimation d'angle — puis `rotate_b64_png` la remet droite. `VisionExtractor._call` extrait ensuite en JSON structuré (`tool_choice` forcé sur `record_transactions`, modèle `claude-sonnet-4-6`, 7 tentatives), avec un **cache versionné** par `prompt_sha` (`extract_cached`) qui ne re-paie que les lots en erreur. Le ticker, absent des formulaires papier, est **ré-associé** en propageant les symboles vus en électronique, complété par une passe LLM (`house/ocr.py:llm_resolve_tickers`). **Point d'architecture** : côté Chambre, la **fusion digital+OCR, le ticker et le secteur se font ANNÉE PAR ANNÉE**, pendant le run (`house/ocr.py:run_ocr_year` écrit directement `06_house_{an}_FINAL.csv`), parce que le volume y est énorme. Le cluster C manuscrit est **exclu par défaut** (dates trop incertaines pour une stratégie datée), avec récupération ciblée des gros déposants.

**Le résultat.** 48 970 lignes OCR. Le volume est très concentré : Rohit Khanna en représente **62,9 %** à lui seul, le top 3 (Khanna, McCaul, Harshbarger) **92,4 %** — ce sont de gros déposants qui déclarent exclusivement sur papier.

## 4.6 La piste papier du Sénat — et pourquoi la fusion y est séparée

**Le problème.** Côté Sénat, le papier est marginal (5 sénateurs) mais bien réel : des images `.gif`. Surtout, contrairement à la Chambre, les données sont *peu nombreuses* — et un dictionnaire de tickers construit année par année y serait trop pauvre.

**Comment on le gère.** `senate/ocr.py` lit les `.gif` par Vision et écrit `06b`. Puis — et c'est là l'**unique vraie asymétrie d'architecture** entre les deux chambres — `senate/fusion.py:main` procède en deux temps : **Étape 1**, fusion non destructrice *par année* (digital + OCR, dédup sur (`natural_key_hash`, `occurrence_index`)) ; **Étape 2**, enrichissement sur le **corpus complet en une seule passe** — `ticker_resolve.resolve_tickers` puis `enrich_sectors` sur *toutes* les années concaténées, avec « dict + LLM partagés, cache unique ». **Pourquoi cette différence avec la Chambre ?** La Chambre a assez de volume pour traiter chaque année isolément ; le Sénat, lui, est petit et chargé en papier — **mutualiser le dictionnaire de tickers sur tout le corpus** (un symbole vu une année sert aux autres) relève nettement la couverture. D'où un module `fusion.py` dédié, exécuté *après* l'assemblage de toutes les années, là où la Chambre fonde et enrichit *pendant* chaque année.

**Le résultat.** 1 680 lignes papier, très concentrées sur Blumenthal (~73 %). C'est ce papier (obligations municipales, *munis*) qui explique la baisse de couverture ticker/secteur du Sénat en 2025–26 (§5).

## 4.7 Donner un secteur à chaque ticker (GICS → ETF SPDR)

**Le problème.** Une stratégie en ETF a besoin du **secteur**, pas seulement du ticker ; or certains tickers sont délistés, fusionnés ou introuvables dans les bases standard.

**Comment on le gère.** `congress_core/sector_enrich.py:enrich_sectors` résout en cascade hybride : `resolve_yfinance` (secteur factuel via `yfinance`) → `repli_resolve_llm` (énumération GICS stricte, pour les tickers que `yfinance` ignore) → overrides manuels, en **traçant qui a tranché** (`sector_source`). Le secteur GICS est mappé 1 :1 vers un **ETF Select Sector SPDR** (Energy XLE, Materials XLB, Industrials XLI, Cons. Disc. XLY, Cons. Staples XLP, Health Care XLV, Financials XLF, IT XLK,



Comm. Services XLC, Utilities XLU, Real Estate XLRE). Au passage, les montants déclarés en **fourchettes** sont ramenés à leur **milieu** (`amounts.py:amount_midpoint`); et l'ancienneté `years_in_office` est calculée hors-ligne puis *appendue* en dernière étape (`enrich_tenure.py`, octet-préservant et idempotent).

**Le résultat.** Secteur renseigné à **83,2 %** (Chambre) et **62,1 %** (Sénat); ancienneté à **100 %** sur les deux chambres.

**Fil rouge.** Notre ticker NVDA est résolu par `yfinance` en secteur *Information Technology*, mappé sur l'ETF XLK; sa fourchette \$1 001–\$15 000 devient `amount_midpoint = 8000,5`.

## 4.8 La table finale : le contrat « 12/12 »

**Le liant — le cœur partagé.** Ce qui assemble tout sans rien perdre, c'est `schema.py` : une **clé naturelle** de 7 champs (`chamber`, `declarant_name`, `transaction_date`, `asset_description`, `operation_type`, `amount_range`, `owner`). Elle exclut *volontairement* le ticker — qui est ajouté *après*, à l'enrichissement; l'inclure rendrait la clé instable. Cette clé est numérotée par `occurrence_index` (le rang d'un lot répété *dans un même dépôt*), si bien que la déduplication (`dedup_canonical`) retire les vrais doublons cross-dépôt mais **préserve les lots multi-comptes réels** — par exemple les déclarations répétées de Khanna via plusieurs comptes, qui sauteraient avec une dédup naïve.

**Le contrat.** La table FINAL compte **28 colonnes** (27 d'assemblage + `years_in_office`). Les **12 champs « métier »** exigés sont *exactement* la constante `schema.CORE_FIELDS` :

```
declarant_name · chamber · party · committee_membership · transaction_date · disclosure_date  
· ticker · sector_gics · etf_proxy · operation_type · amount_midpoint · asset_type
```

**Nuance d'honnêteté — « présents » n'est pas « sans valeurs manquantes ».** Les 12 champs sont **présents** dans le schéma des deux chambres. Mais trois ont des **trous légitimes**, par nature et non par défaut d'extraction : `ticker/sector_gics/etf_proxy` sont vides pour les actifs non cotés (obligations, *munis*, fonds privés — il n'existe pas de symbole à leur attribuer); et `committee_membership` est un **snapshot courant**, pas l'historique daté. Les champs structurellement toujours renseignables (nom, chambre, parti, dates, opération, montant) sont, eux, quasi-complets. Nous assumons cette distinction plutôt que de la masquer. Les taux de remplissage exacts (par champ et par an) sont en annexe 12.3.

**Fil rouge.** Notre ligne sort complète : G000596, Greene, house, ..., 2024-12-24, ..., NVDA, NVIDIA Corp, Stock, Information Technology, XLK, Purchase, ..., 8000.5, ... — une ligne de `06_house_2024_FINAL.csv`, prête pour l'analyse.

## 5 Le rapport de qualité

**Le problème.** Une table « pleine » n'est pas une table *saine* : les dates peuvent être incohérentes, les délais légaux non respectés, la couverture concentrée sur quelques personnes.

**Comment on l'obtient.** `congress_core/quality.py` (`build_report` → `_figures`) calcule **5 contrôles** et génère 4 figures — **en lecture seule des FINAL, sans aucun appel API** (matplotlib Agg). (a) **Cohérence des dates** (`disclosure ≥ transaction`) : 99,8 % Chambre, 100,0 % Sénat. (b) **Délai légal** : 86,9 % (Chambre) et 84,9 % (Sénat) des dépôts dans les 45 jours (médiane **28 j**). (c) **Montants** : médiane **8 000 \$**, quartile haut **32 500 \$**. (d) **Couverture par membre** : 320 déposants, dont **206** avec  $\geq 10$  transactions (éligibles) et **150** actifs sur  $\geq 3$  années. (e) **Achats sans sortie déclarée** : 22,5 % (Chambre) / 39,6 % (Sénat). Le détail par an est en annexe 12.3.

**La couverture par an — et la baisse Sénat 2025–26.** Le global masque une dynamique : au Sénat, la couverture ticker chute à **45,4 %** (2025) et **43,7 %** (2026), contre  $\sim 80\%$  les années antérieures; le secteur suit. **Pourquoi ?** La part de papier OCR et surtout d'**obligations municipales** (non cotées, donc `ticker=None` *légitimement*, ex. Blumenthal) y augmente fortement. Ce n'est pas une dégradation d'extraction mais un changement de composition des actifs; le global (71,4 % / 62,1 %) est tiré vers le haut par les années antérieures. La fiabilité des dates OCR est suivie par `date_confidence` (fenêtre 75 j,

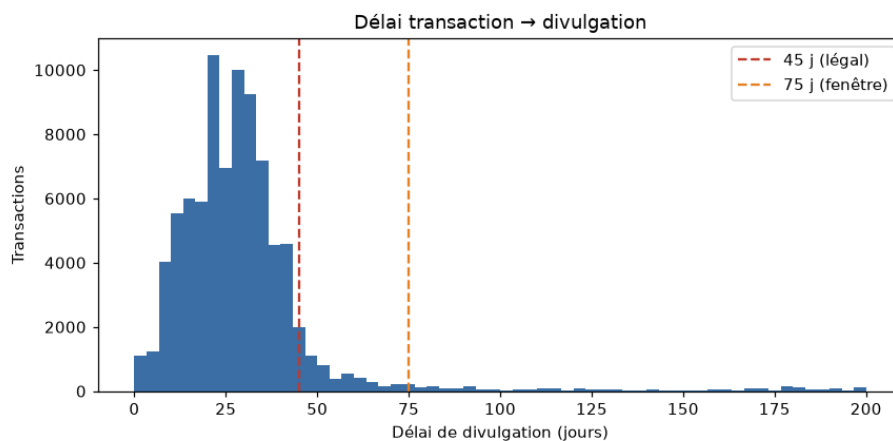


FIGURE 1 – Délai disclosure – transaction (0–200 j), lignes 45 j (légal) / 75 j (fenêtre). Générée par `congress_core/quality.py` (`_figures`, source `delay_buckets`) — lecture seule, aucun appel API.

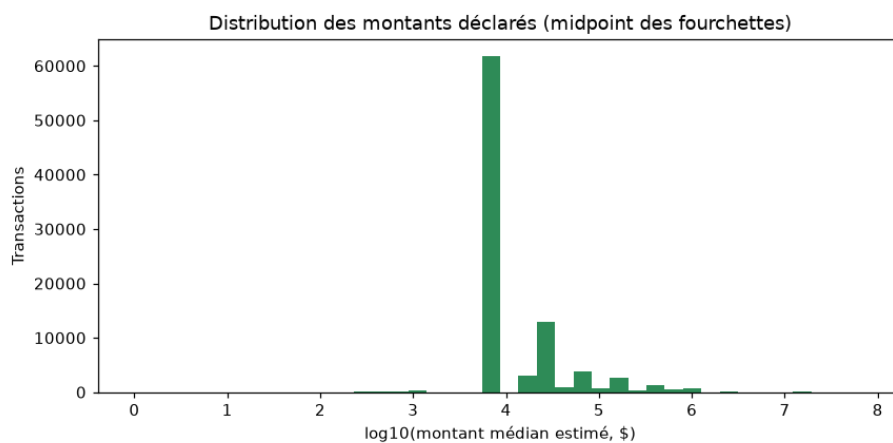


FIGURE 2 – Distribution de `amount_midpoint` en échelle `log10`. Générée par `quality.py` (`_figures`, source `amount_distribution`).

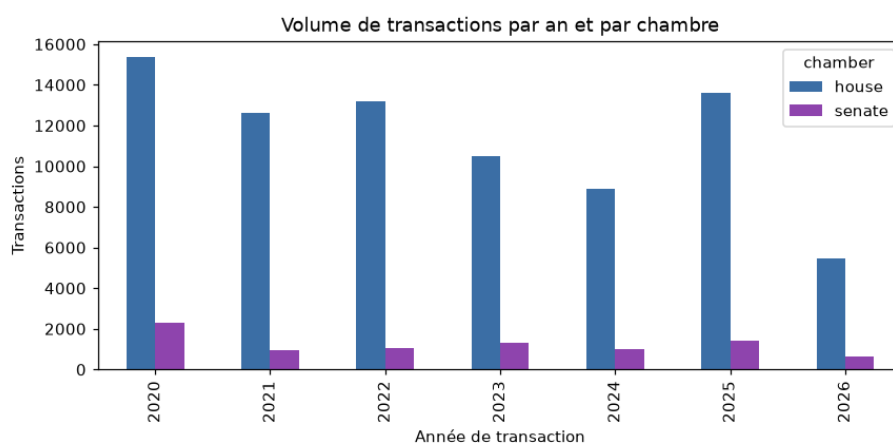


FIGURE 3 – Transactions par **année de transaction** et par chambre. Générée par `quality.py` (`_figures`).



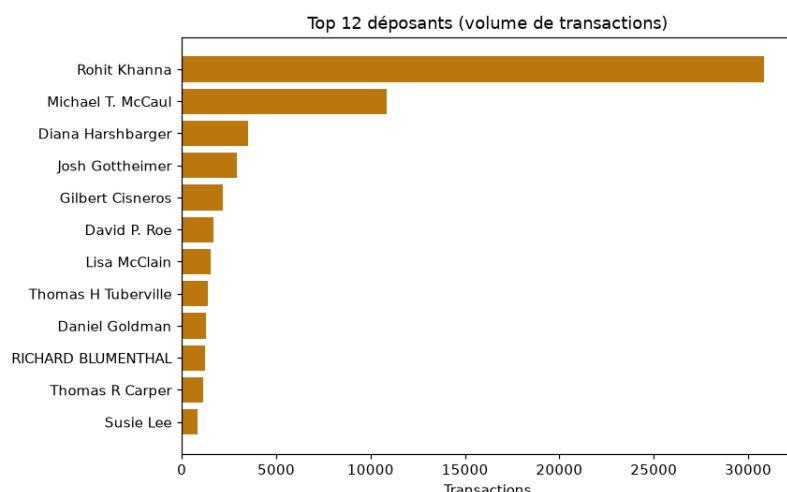


FIGURE 4 – Top 12 déposants par **nombre de transactions**. Générée par `quality.py` (`_figures`, source `coverage_per_member`) — la domination des déposants-papier (Khanna, McCaul) y est visible.

colonne *OCR seulement*) : 95,3 % plausibles côté Chambre — le creux 2021 (79,6 %) est entièrement Diana Harshbarger (manuscrit dense), anomalie localisée, non systémique.

## 6 La validation externe (Quiver)

**Le problème.** Comment savoir si l'extraction est juste, *sans* se contenter de se croire ?

**Comment on le gère.** On compare le FINAL à Quiver sur **deux axes**, sans jamais le réinjecter : (1) **comptes par déposant** (per-lot) ; (2) **recouvrement transaction-niveau** via la clé bioguide | ticker | date | type (la date appariée est le *Traded* de Quiver). La **couverture** = transactions Quiver retrouvées / total Quiver.

**Le résultat** — **Chambre** (clé « brute », sans tolérance de date, `quiver.validate_quiver_house`) : couverture globale **85,9 %** (43 136 / 50 224), par année de 80,7 % à 91,3 %. C'est un **plancher** : la clé sur date brute sous-estime sur les dates OCR bruitées. Les **249 « manquants bruts »** se réduisent à **9 vrais-absents** après explication des artefacts (tolérance  $\pm 1$  j, amendements, options/obligations) — **aucun bug d'extraction**. **Sénat** (clé normalisée, `quiver.audit.reconcile`) : couverture **98–100 %/an**, accord de sens  $\sim 95,7$  %, montant  $\sim 93,2$  %, **554 amendements** dédoublés, **0 vrai raté** après audit adversarial. Le détail par an est en annexe 12.3.

**Le constat « OCR = source unique ».** Quiver — comme Kadoa et *House Stock Watcher* — **ne voit aucun scan** : 0 ligne pour les gros déposants-papier (Khanna, Harshbarger, Blumenthal). Le terme `only_ours` de la Chambre ( $\sim 16\,000$  transactions) est donc surtout le **papier que les agrégateurs ne voient pas** — pas une erreur, mais notre **valeur ajoutée**, validée en interne (cohérence, stabilité run-à-run, `date_confidence`). `crosscheck.py` matérialise ce statut par déposant (`quiver-validable` / `ocr_unique` / `digital` ; détail en annexe 12.3).

## 7 Assurance qualité & reproductibilité

**Le problème.** Le pipeline n'est **pas rejouable hors-ligne** (le scraping exige le réseau, l'OCR l'API) ; rejouer une année digitale produirait une table *vide* (les PDF lisibles ont été parsés puis écartés).

**Comment on le gère.** On prouve la correction par **reproduction depuis les colonnes figées**, preuve plus forte car déterministe : (i) un **golden octet-à-octet** — empreintes SHA-256 de toutes les sorties, **108** fichiers Chambre et **69** Sénat (`check_golden.py` / `senate_check_golden.py` doivent afficher « ZÉRO ÉCART ») ; (ii) des **reproductions fonction-par-fonction** (`test_senate_repro` reconstruit `natural_key_hash` 8 841/8 841, l'identité 67/67, le ticker 65/65 ; côté Chambre `test_schema/test_`

amounts\_tickers/ test\_identity/test\_tenure); (iii) test\_vision\_sha/test\_incremental (le prompt\_sha est stable, un 2<sup>e</sup> run OCR fait 0 appel); (iv) audit\_metrics.py, qui recompute toutes les métriques et *assure* les invariants — la source de vérité chiffrée de ce rapport.

## 8 Hypothèses explorées puis révisées

La valeur « recherche » tient autant aux choix *révisés* qu’aux résultats. Pour chacun : **hypothèse de départ** → **problème constaté** → **décision finale**.

**Échantillon d’abord.** Avant le run complet des 547 PDF, house/echantillon.py a OCR-isé un **échantillon stratifié** des clusters A/B/C pour *mesurer* la qualité et durcir le prompt — plutôt que de découvrir les modes d’échec en production.

**Deskew OCR.** *Départ* : demander au modèle « de combien faut-il tourner ? ». *Problème* : tâche spatiale peu fiable — il répond presque toujours « 90 ». *Décision* : lui montrer la page aux **4 rotations** et lui faire *reconnaître* la droite. La concordance OCR passe de **59,7 %** à **≈69 %**.

**Déduplication.** *Départ* : dédupliquer sur la seule clé naturelle. *Problème* : cela écrase des **lots multi-comptes réels** (Khanna déclarant plusieurs fois le même titre). *Décision* : occurrence\_index → dédup **non destructrice** sur le couple.

**Cluster C manuscrit.** Tenté, puis **exclu** par défaut (dates trop incertaines pour une stratégie datée). Une sonde **Opus vs Sonnet** sur C a confirmé qu’Opus **ne corrige pas le goulot** (les dates), pour un coût  $\approx \times 3$  — on garde Sonnet, C reste exclu.

**Fenêtre de plausibilité.** Le STOCK Act vise  $\approx 45$  j ; on tolère **75 j** (date\_confidence) pour absorber le bruit OCR — colonne calculée *sur l’OCR seulement*.

**Ticker en trois voies.** Ajoutées par échecs successifs : explicite, dictionnaire électronique, puis LLM (filtre is\_equity, pour ne *jamaïs* tickeriser une obligation/muni).

**Quiver, deux régimes de clé.** Chambre = clé « brute » (couverture = plancher); Sénat = clé « normalisée » (reconcile) avec dédup des amendements. On *n’unifie pas* les clés : chaque chambre garde sa sémantique exacte.

**Per-année (Chambre) vs corpus (Sénat).** L’enrichissement se fait *pendant* chaque année côté Chambre (volume), mais *sur tout le corpus en une passe* côté Sénat (pour mutualiser le dictionnaire de tickers) — cf. §4.

## 9 Limites (honnêteté)

- **Cluster C manuscrit exclu par défaut** (dates trop incertaines) — choix assumé, pas une lacune cachée.
- **Validation papier externe impossible** : aucun agrégateur ne voit les scans → pas de taux d’exactitude *externe* sur le papier (seulement des contrôles internes).
- **Appariement Quiver Chambre sans tolérance de date** : la couverture **85,9 %** est un **plancher**, pas une borne haute.
- **ticker/sector\_gics/etf\_proxy structurellement absents** pour munis et obligations (actifs non cotés) — par nature, pas par défaut.
- **Commissions non « point-in-time »** : committee\_membership est un *snapshot* courant, pas l’historique daté.
- **date\_confidence = heuristique binaire** (fenêtre 75 j), **colonne OCR seulement** ; ce n’est pas une vérité-terrain.

## 10 Périmètre & suite

La **couche données** décrite ici est livrée : 90 487 transactions extraites, identifiées, enrichies (table 12/12), validées et reproductibles. La **stratégie et le backtest** associés (copy-trading actions puis ETF sectoriels) constituent un travail de recherche séparé, hors-périmètre de ce rapport.

## 11 Conclusion

Sur deux chambres et sept ans, le pipeline reconstruit **90 487** transactions à partir des sources officielles, en traitant honnêtement les deux pistes (électronique déterministe *et* scannée par OCR Vision). Chaque ligne est identifiée (99,99 / 100 %), enrichie (table 12/12 sur les deux chambres) et validée contre Quiver sans jamais le réinjecter (85,9 % Chambre, 98–100 % Sénat ; 9 et 0 vrais-absents). Le papier scanné, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne ; la correction est figée (golden octet-à-octet) et reproductible. Surtout, le rapport assume ses nuances — « présents » n’est pas « sans manquants », la couverture Sénat 2025–26 baisse pour des raisons de composition d’actifs, et la couverture Quiver Chambre est un plancher. C’est une couche données sur laquelle une stratégie peut être bâtie *en connaissant ses limites*.

## 12 Annexes

### 12.1 Annexe A — modules & fonctions clés

Pour chaque module : son rôle, et ses **fonctions centrales** (les helpers privés mineurs sont omis). Le pipeline complet vit dans 27 modules de code (3 couches) ; l’orchestration se lit dans `pipeline.py:build_steps`, puis `house/digital.py:run_year`, `house/ocr.py:run_ocr_year`, `senate/digital.py:run_year` et `senate/fusion.py:main`.

| Module                                          | Rôle                  | Fonctions clés                                                                                                                                                                                                              |
|-------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>congress_core/</b> — boîte à outils partagée |                       |                                                                                                                                                                                                                             |
| <code>schema.py</code>                          | clé naturelle + dédup | <code>natural_key_hash</code> (SHA-256 de 7 champs) · <code>add_occurrence_index</code> · <code>dedup_canonical</code> (non destructrice)                                                                                   |
| <code>identity.py</code>                        | nom → bioguide + méta | <code>load_reference</code> · <code>make_matcher</code> (override→exact+surnoms→nom unique) · <code>enrich_identity</code> · <code>add_years_in_office</code>                                                               |
| <code>amounts.py</code>                         | montant & opération   | <code>amount_midpoint</code> (milieu de fourchette) · <code>operation_type_from_code</code> (P/S/E)                                                                                                                         |
| <code>tickers.py</code>                         | ticker + type d’actif | <code>explicit_ticker</code> · <code>recover_ticker</code> · <code>infer_asset_type</code> (par chambre)                                                                                                                    |
| <code>vision_ocr.py</code>                      | OCR Vision + cache    | <code>detect_rotation</code> (deskew 4 rotations) · <code>pdf_to_b64_images</code> · <code>VisionExtractor.prompt_sha</code> · <code>extract_cached</code> (reprise au batch)                                               |
| <code>llm_resolve.py</code>                     | cache LLM nom→ticker  | <code>resolve_names_to_tickers</code> · <code>VersionedLLMCache</code> · <code>map_batch</code> (tool_use + backoff)                                                                                                        |
| <code>quiver.py</code>                          | validation externe    | <code>fetch_quiver</code> · <code>validate_quiver_house</code> (clé brute) · <code>reconcile</code> (clé normalisée)                                                                                                        |
| <code>crosscheck.py</code>                      | statut par déposant   | <code>per_filer_status</code> · <code>add_external_counts</code> (Kadoda/HSW) · <code>summary</code>                                                                                                                        |
| <code>sector_enrich.py</code>                   | secteur GICS → ETF    | <code>resolve_yfinance</code> · <code>resolve_llm</code> · <code>enrich_sectors</code> (+ <code>sector_source</code> )                                                                                                      |
| <code>reporting.py</code>                       | sorties & QA          | <code>qa_flags</code> · <code>upsert_status</code> · <code>write_excel</code>                                                                                                                                               |
| <code>paths.py</code>                           | chemins & secrets     | <code>DataRoot</code> (data/{chambre}) · <code>get_secret</code> / <code>load_env</code>                                                                                                                                    |
| <code>pipeline.py</code>                        | orchestrateur         | <code>build_steps</code> (séquence 6 étapes) · <code>main</code> (sous-processus, -dry-run)                                                                                                                                 |
| <code>quality.py</code>                         | rapport qualité       | <code>load_final</code> · <code>date_coherence</code> · <code>delay_buckets</code> · <code>amount_distribution</code> · <code>coverage_per_member</code> · <code>unmatched_purchase_rate</code> · <code>build_report</code> |

| Module                                | Rôle                   | Fonctions clés                                                                                                                                                                                                                            |
|---------------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>enrich_tenure.py</code>         | ancienneté             | <code>enrich_file</code> (append years_in_office octet-préservant) · <code>main</code>                                                                                                                                                    |
| <b>house/ — orchestrateur Chambre</b> |                        |                                                                                                                                                                                                                                           |
| <code>digital.py</code>               | PTR électroniques      | <code>load_ptr_index</code> (XML, FilingType=P) · <code>build_manifest</code> (lisible/non) · <code>parse_ptr</code> (TXN_RE) · <code>join_identity</code> · <code>finalize</code> · <code>validate_quiver</code> · <code>run_year</code> |
| <code>ocr.py</code>                   | scans → FINAL (par an) | <code>extract_from_pdf</code> (Vision) · <code>llm_resolve_tickers</code> · <code>normalize</code> (+ <code>date_confidence</code> ) · <code>run_ocr_year</code> (fusion+ticker+secteur)                                                  |
| <code>echantillon.py</code>           | pilote OCR             | <code>run_echantillon</code> (stratifié) · <code>compare_quiver</code>                                                                                                                                                                    |
| <b>senate/ — orchestrateur Sénat</b>  |                        |                                                                                                                                                                                                                                           |
| <code>digital.py</code>               | eFD électronique       | <code>accept_agreement</code> (CSRF) · <code>fetch_ptr_list</code> (report_types=[11]) · <code>parse_electronic</code> (pd.read_html) · <code>run_year</code>                                                                             |
| <code>ocr.py</code>                   | papier .gif → 06b      | <code>discover_index</code> · <code>ocr_one</code> (un .gif → Vision) · <code>build_table</code>                                                                                                                                          |
| <code>ocr_engine.py</code>            | moteur OCR figé        | <code>extract_report</code> · <code>normalize</code> · <code>_fix_year</code>                                                                                                                                                             |
| <code>fusion.py</code>                | fusion + enrich corpus | <code>main</code> (Étape 1 fusion/an, Étape 2 enrichissement corpus) · <code>date_confidence</code>                                                                                                                                       |
| <code>identity.py</code>              | référentiel Sénat      | <code>load_reference</code> · <code>make_matcher</code> (désambiguïsation chambre) · <code>enrich</code> (bioguide+ticker+dédup+schéma)                                                                                                   |
| <code>ticker_resolve.py</code>        | ticker 3 voies         | <code>resolve_tickers</code> (explicit→dict→LLM) · <code>llm_resolve_tickers</code>                                                                                                                                                       |
| <code>sector_enrich.py</code>         | secteur Sénat          | <code>resolve_sectors</code> · <code>enrich_sectors</code>                                                                                                                                                                                |
| <code>quiver_audit.py</code>          | réconciliation         | <code>reconcile</code> (07c-07f)                                                                                                                                                                                                          |
| <code>revalidate_quiver.py</code>     | re-validation offline  | <code>main</code>                                                                                                                                                                                                                         |
| <code>census_probe.py</code>          | Phase 0 (census)       | <code>parse_electronic</code> · <code>main</code> (census + sondage parser)                                                                                                                                                               |

Les 3 `__init__.py` (exports de paquet) et `tests/regression/` (golden + reproductions) complètent les 44 fichiers `.py` du dépôt.

## 12.2 Annexe B — le schéma FINAL (28 colonnes)

| #  | Colonne                           | Sens                                     | Remplie par                                     |
|----|-----------------------------------|------------------------------------------|-------------------------------------------------|
| 1  | <code>bioguide_id</code>          | identifiant officiel du membre           | <code>identity</code>                           |
| 2  | <code>declarant_name</code>       | nom tel que déposé                       | <code>parse</code> / <code>OCR</code>           |
| 3  | <code>chamber</code>              | <code>house</code> / <code>senate</code> | <code>enrich_identity</code>                    |
| 4  | <code>party</code>                | parti                                    | <code>Reference.party</code>                    |
| 5  | <code>state_district</code>       | état + district                          | <code>03_ptr_index</code>                       |
| 6  | <code>committee_membership</code> | commissions (liste)                      | <code>Reference.committees</code>               |
| 7  | <code>committees_key_flag</code>  | commission « clé » ?                     | <code>is_key_committee</code>                   |
| 8  | <code>transaction_date</code>     | date de la transaction                   | <code>parse</code> / <code>OCR</code>           |
| 9  | <code>disclosure_date</code>      | date de divulgation (dépôt)              | <code>03_ptr_index</code>                       |
| 10 | <code>ticker</code>               | symbole boursier                         | <code>tickers</code> / <code>llm_resolve</code> |
| 11 | <code>asset_description</code>    | libellé de l'actif                       | <code>parse</code> / <code>OCR</code>           |
| 12 | <code>asset_type</code>           | type d'actif                             | <code>infer_asset_type</code>                   |
| 13 | <code>sector_gics</code>          | secteur GICS                             | <code>sector_enrich</code>                      |
| 14 | <code>etf_proxy</code>            | ETF SPDR du secteur                      | <code>GICS_TO ETF</code>                        |
| 15 | <code>operation_type</code>       | Purchase / Sale / ...                    | <code>operation_type_from_code</code>           |

| #  | Colonne           | Sens                              | Remplie par          |
|----|-------------------|-----------------------------------|----------------------|
| 16 | amount_range      | fourchette déclarée               | parse / OCR          |
| 17 | amount_midpoint   | milieu de la fourchette           | amount_midpoint      |
| 18 | amount_split_flag | lot multi-actifs ?                | finalize             |
| 19 | owner             | propriétaire (self/spouse...)     | normalize            |
| 20 | doc_id            | identifiant du PTR                | 03_ptr_index         |
| 21 | source_url        | URL de la source                  | 03_ptr_index         |
| 22 | natural_key_hash  | clé de dédup (7 champs)           | schema               |
| 23 | occurrence_index  | rang du lot intra-dépôt           | add_occurrence_index |
| 24 | provenance        | *-electronic / *-ocr              | pipeline             |
| 25 | date_confidence   | plausible / implausible (75 j)    | date_confidence      |
| 26 | ticker_source     | explicit / elec_dict / llm / none | tickers / llm        |
| 27 | sector_source     | yfinance / llm / manual           | sector_enrich        |
| 28 | years_in_office   | ancienneté à la date du trade     | enrich_tenure        |

L'ordre exact diffère légèrement entre Chambre et Sénat (*date\_confidence* plus tôt au Sénat); le contenu garanti — les 12 champs « métier » — est identique.

### 12.3 Annexe C — métriques vérifiées (recalculées depuis les FINAL)

Lue après le rapport, cette annexe dit si l'extraction a bien marché. Tous les chiffres sont recomputés depuis les tables FINAL par `tests/regression/audit_metrics.py` et `congress_core/quality.py`.

#### C.1 — Volumes par an.

| An          | Chambre       |               |               |             | Sénat        |              |              |            |
|-------------|---------------|---------------|---------------|-------------|--------------|--------------|--------------|------------|
|             | dig.          | OCR           | FINAL         | dépos.      | dig.         | OCR          | FINAL        | sén.       |
| 2020        | 6 886         | 8 791         | 15 677        | 109         | 1 706        | 255          | 1 961        | 33         |
| 2021        | 5 457         | 6 816         | 12 273        | 120         | 1 098        | 281          | 1 379        | 35         |
| 2022        | 3 601         | 10 900        | 14 501        | 110         | 919          | 182          | 1 101        | 27         |
| 2023        | 4 161         | 6 329         | 10 490        | 100         | 1 062        | 97           | 1 159        | 30         |
| 2024        | 2 694         | 6 277         | 8 971         | 97          | 946          | 84           | 1 030        | 24         |
| 2025        | 7 577         | 6 067         | 13 644        | 107         | 943          | 478          | 1 421        | 31         |
| 2026        | 2 300         | 3 790         | 6 090         | 84          | 487          | 303          | 790          | 22         |
| <b>Tot.</b> | <b>32 676</b> | <b>48 970</b> | <b>81 646</b> | <b>256*</b> | <b>7 161</b> | <b>1 680</b> | <b>8 841</b> | <b>64*</b> |

\* bioguides *uniques* sur 7 ans (la somme annuelle compte un élu chaque année).

#### C.2 — Couverture des champs enrichis, par an (% renseigné).

| An           | Chambre     |             |             |             | Sénat       |             |             |             |
|--------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|              | ticker      | sect.       | a.type      | comm.       | ticker      | sect.       | a.type      | comm.       |
| 2020         | 86,5        | 83,4        | 91,0        | 64,9        | 88,1        | 80,5        | 96,3        | 34,5        |
| 2021         | 87,3        | 85,8        | 90,1        | 78,7        | 80,9        | 70,1        | 94,2        | 54,3        |
| 2022         | 87,1        | 85,2        | 91,0        | 89,2        | 79,7        | 65,3        | 96,1        | 78,3        |
| 2023         | 78,9        | 76,6        | 88,6        | 96,8        | 77,7        | 69,5        | 96,3        | 71,4        |
| 2024         | 81,3        | 78,3        | 87,4        | 93,6        | 68,0        | 59,2        | 100         | 77,8        |
| 2025         | 87,8        | 86,3        | 94,8        | 97,6        | <b>45,4</b> | <b>37,2</b> | 100         | 84,9        |
| 2026         | 85,4        | 84,4        | 94,9        | 99,7        | <b>43,7</b> | <b>35,7</b> | 100         | 86,2        |
| <b>Glob.</b> | <b>85,3</b> | <b>83,2</b> | <b>91,1</b> | <b>86,6</b> | <b>71,4</b> | <b>62,1</b> | <b>97,3</b> | <b>65,6</b> |

### C.3 — Les 5 contrôles qualité (global).

| Contrôle                                                       | Chambre              | Sénat   |
|----------------------------------------------------------------|----------------------|---------|
| (a) dates cohérentes ( <code>disclosure ≥ transaction</code> ) | 99,8 %               | 100,0 % |
| (b) dépôts $\leq 45$ j (délai médian 28 j)                     | 86,9 %               | 84,9 %  |
| (c) montants — médiane / quartile haut                         | 8 000 \$ / 32 500 \$ |         |
| (d) déposants ( $\geq 10$ trades / $\geq 3$ ans)               | 320 (206 / 150)      |         |
| (e) achats sans sortie déclarée                                | 22,5 %               | 39,6 %  |

### C.4 — Concordance Quiver Chambre, par an (clé brute ; couverture = plancher).

| An           | couv. %     | Quiver retrouvées | only_quiver  | vrais-absents  |
|--------------|-------------|-------------------|--------------|----------------|
| 2020         | 85,0        | 7 106             | 1 253        | 16             |
| 2021         | 88,6        | 5 221             | 672          | 14             |
| 2022         | 80,7        | 6 061             | 1 452        | 29             |
| 2023         | 81,0        | 6 128             | 1 442        | 65             |
| 2024         | 85,8        | 4 670             | 772          | 40             |
| 2025         | 91,3        | 9 835             | 938          | 58             |
| 2026         | 88,0        | 4 115             | 559          | 27             |
| <b>Glob.</b> | <b>85,9</b> | <b>43 136</b>     | <b>7 088</b> | <b>249 → 9</b> |

Accord de sens et de montant sur les transactions appariées : ~93 % (deux mesures).

### C.5 — Concordance Quiver Sénat, par an (clé normalisée, reconcile).

| An          | couv. %       | sens %       | date %       | montant %    | amend. dédup. | only_quiver        |
|-------------|---------------|--------------|--------------|--------------|---------------|--------------------|
| 2020        | 99,4          | 95,7         | 99,4         | 93,2         | 144           | 8                  |
| 2021        | 98,0          | 87,0         | 98,0         | 86,1         | 75            | 8                  |
| 2022        | 99,4          | 91,7         | 99,4         | 91,5         | 86            | 4                  |
| 2023        | 99,7          | 93,0         | 99,7         | 89,5         | 94            | 2                  |
| 2024        | 99,6          | 95,6         | 99,6         | 92,8         | 111           | 2                  |
| 2025        | 99,8          | 96,4         | 99,8         | 96,1         | 20            | 1                  |
| 2026        | 100,0         | 100,0        | 100,0        | 99,3         | 24            | 0                  |
| <b>Tot.</b> | <b>98–100</b> | <b>~95,7</b> | <b>~99,4</b> | <b>~93,2</b> | <b>554</b>    | <b>0 vrai raté</b> |

### C.6 — Fiabilité des dates OCR (`date_confidence`, lignes OCR seulement).

| An          | Chambre (OCR) |              | Sénat (OCR) |              |
|-------------|---------------|--------------|-------------|--------------|
|             | plausible %   | implausibles | plausible % | implausibles |
| 2020        | 98,6          | 120          | 96,1        | 9            |
| 2021        | <b>79,6</b>   | <b>1 389</b> | 97,5        | 6            |
| 2022        | 96,4          | 394          | 99,5        | 0            |
| 2023        | 99,2          | 50           | 100,0       | 0            |
| 2024        | 96,5          | 220          | 97,6        | 0            |
| 2025        | 98,1          | 118          | 99,0        | 3            |
| 2026        | 100,0         | 0            | 99,7        | 0            |
| <b>Tot.</b> | <b>95,3</b>   | <b>2 291</b> | <b>98,5</b> | <b>18</b>    |

Le creux 2021 Chambre est entièrement Diana Harshbarger (2 documents manuscrits denses).

### C.7 — Clusters OCR & concentration.



| Clusters Chambre (census 547 scans) | Concentration des déposants OCR                             |
|-------------------------------------|-------------------------------------------------------------|
| A — tapé droit : 74 docs            | Chambre : Khanna 30 862 (62,9 %), McCaul 10 876 (22,2 %),   |
| B — tapé tourné : 322 docs          | Harshbarger 3 514 (7,2 %) → top 3 = 92,4 %, top 10 = 99,0 % |
| C — manuscrit : 151 docs (exclu)    | Sénat : Blumenthal 1 233 (73 %), Boozman 379, Burr 52,      |
|                                     | Feinstein 14, Fetterman 2                                   |

## C.8 — Statut de validation par déposant (Chambre, crosscheck).

| Statut                                                   | déposants | transactions | dont OCR |
|----------------------------------------------------------|-----------|--------------|----------|
| quiver_validable (Quiver le couvre)                      | 212       | 78 530       | 47 004   |
| ocr_unique (source unique : Quiver = 0, OCR $\geq$ 50 %) | 13        | 1 957        | 1 949    |
| digital (peu / pas d'OCR)                                | 31        | 1 155        | 13       |

Identité : Chambre 256 bioguides / 275 noms (99,99 %), Sénat 64 / 67 (100 %). Golden (zéro écart) : 108 fichiers Chambre, 69 Sénat. **Total : 90 487 transactions**, 2020–2026.

## 12.4 Annexe D — reproduire / lancer

```
# Installation
python3.12 -m venv .venv && ./venv/bin/pip install -e .

# Pipeline complet (reseau + API Vision requis) ; --dry-run = voir la sequence
./venv/bin/python -m congress_core.pipeline --years 2020-2026
./venv/bin/python -m congress_core.pipeline --years 2024 --dry-run

# Rapport de qualite (offline, lecture seule des FINAL)
# -> docs/RAPPORT_QUALITE.md + docs/quality/*.png
./venv/bin/python -m congress_core.quality

# Filet "zero changement" (doit afficher ZERO ECART)
./venv/bin/python tests/regression/check_golden.py          # Chambre, 108
./venv/bin/python tests/regression/senate_check_golden.py   # Senat,    69
./venv/bin/python tests/regression/test_senate_repro.py     # reproductions
```

*Document dérivé et vérifié sur le code source réel et les tables FINAL du dépôt. Toute divergence avec une autre documentation doit être tranchée en faveur du code et des données. Quiver = vérification externe, jamais réinjecté.*